

Towards Multi-Modal Context-Augmented Embodied Agent for Interactive BIM Space Layout Modification

Suhyung Jang^{1,2}, Gaang Lee³, and André Borrmann^{1,2}

¹Georg Nemetschek Institute Artificial Intelligence for the Built World, Technical University of Munich, Munich, Bayern, Germany

²Chair of Computing in Civil and Building Engineering, School of Design and Engineering, Technical University of Munich, Munich, Germany

³Assistant Professor, Civil and Environmental Engineering Department, Faculty of Engineering, University of Alberta

suhyung.jang@tum.de, gaang@ualberta.ca, andre.borrmann@tum.de

Abstract: Generative artificial intelligence (AI) is increasingly explored in architectural design, especially with image-driven space layout generation. However, pixel-based image outputs lack the operational structure for building information modeling (BIM), and iterative workflows involving repeated image–BIM conversions often degrade fidelity and lose the structured context required for constraint-aware modification. Meanwhile, topology-based structured representations often lack the intuitive visual grounding necessary for accurate spatial reasoning, causing misalignment between user intent and system observations. This paper introduces a preliminary study on a multi-modal context–augmented embodied agent method for interactive BIM space layout modification that bridges image-centric reasoning and executable BIM editing. The method grounds the agent using two aligned input contexts. It provides a java script object notation (JSON) topology state with stable element identifiers (IDs) and geometry placeholders, and a floorplan view annotated with the same IDs generated from the JSON. Following an interpret–fill–match–structure–execute–check framework, the relocation agent produces schema-constrained boundary relocation states, while an evaluation agent verifies requirement and constraint satisfaction to support iterative refinement. We validate the proposed method on 15 single-floor residential layouts using two space layout modification tasks that vary in constraints. Compared to a topology-only baseline, augmenting topology with the annotated floorplan image increases relocation success rate from 26.7% to 46.7%. The results highlight the practical value of aligning structured BIM representations with task-relevant visual context for robust, closed-loop BIM editing.

1. Introduction

Recent use of generative artificial intelligence (AI) in architectural design has rapidly expanded, with strong focus on early-stage, image-based generation (Jang, Roh and Lee, 2025; Choi *et al.*, 2024; Cao, Abdul Aziz and Mohd Arshard, 2025). As general image–text understanding and generation capabilities have improved, generative AIs increasingly show strong performance in understanding architectural image context and synthesizing plausible design alternatives. Building layout generation has become a particularly active research direction in prior studies.

However, relying only on image-based input and output introduces a critical integration barrier in building information modeling (BIM) workflows. Pixel-based image outputs require a processing step to be operational in BIM authoring. They require instantiation into BIM elements and further

enrichment of geometric and semantic information. Moreover, practical layout modification is inherently iterative, which commonly requires repeated conversions from BIM models into images and reconstruction of BIM models from images for further elaboration. In such image–BIM round trips, semantic and geometric fidelity can degrade, hindering a streamlined AI-augmented BIM workflow.

Meanwhile, a growing body of work on integrating large language models (LLMs) into BIM systems focuses on manipulating structured BIM data, either wholly (e.g., directly on IFC-based representations) or partially (e.g., via curated data structures and tool chains). While these approaches expand the scope of natural-language interaction with BIM systems, they often rely on text-represented structured data. Such multi-hierarchical textual representations can become difficult to interpret as relational complexity increases, making it difficult for AI models to understand specific target elements and required actions for modification. From the AI models’ perspective, text-structured representation lacks intuitive context that users are looking at while giving modification instructions, which can easily lead to misalignment between user intent and the intent interpreted by the system. In particular, prior work focuses insufficient attention on how users formulate modification requests grounded in what they see on screen, and to how a system can align its internal observations with that visual context.

To address this, we present a preliminary study on a multi-modal context–augmented embodied agent framework for interactive BIM space layout modification. The key idea is to provide generative AI with aligned visual context and space layout topology extracted from BIM models on rule-based, and to realize modification as an embodied agent that operates within the BIM authoring environment with iterative planning and verification. For example, in our testing scenario of space layout modification, a user request such as “expand the living room” is first interpreted in relation to the existing floorplan topology (Figure 1). The system then identifies implicit and explicit tasks and constraints from the request, structuring them into executable BIM actions, and refines the result through interpretation and reflection.

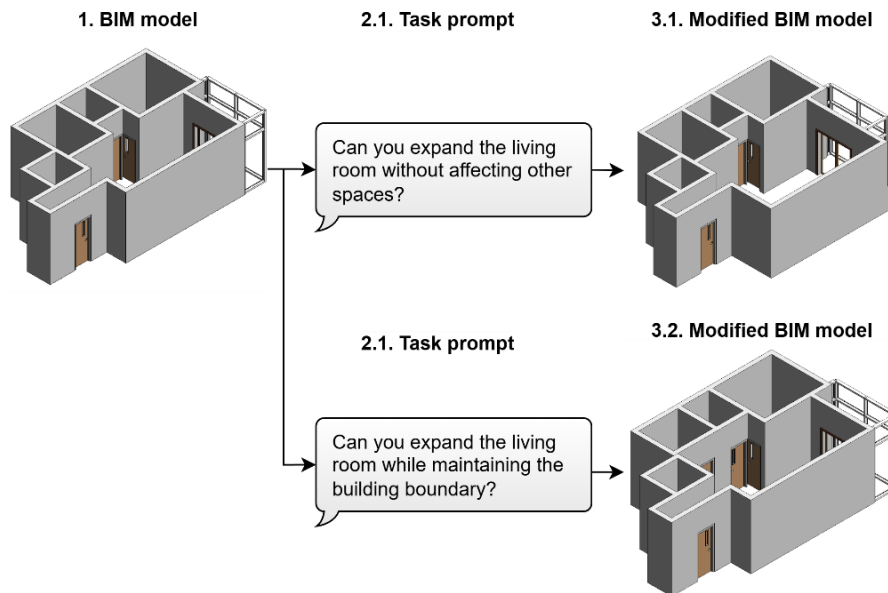


Figure 1. Conceptual illustration of interactive BIM space layout modification.

The proposed framework follows our previously proposed generalized generative-AI-augmented BIM framework (i.e., interpret–fill–match–structure–execute–check) (Lee, Jang and Hyun, 2024). In our space layout modification scenario, each step of framework corresponds to the following sequence. The agent interprets user prompts to infer intent, fills missing task context using aligned multimodal observations (structured topology and element-annotated floorplan views), matches the intent to

implicit (e.g., no overlaps between spaces) and explicit constraints (e.g., expand without affecting other spaces), structures the requested change into executable BIM authoring actions (e.g., element relocation), executes these actions in the BIM environment, and checks the updated state through re-perception and constraint verification to enable iterative refinement. The focus of this study is to propose a method to embed visual context along with text prompt for interpretation step and validate if such visual context improves task execution performance of generative-AI-augmented BIM systems.

The remainder of this paper is organized as follows. The second section discusses the motivation for this study. The third section describes the proposed framework. The fourth section explains how the proposed framework is validated. The fifth section outlines the results of the validation. Finally, the sixth section provides final remarks on the findings and proposes directions for future research.

2. Literature Review

This section reviews trends in image-based space layout generation, how recent generative-AI-augmented BIM systems represent BIM context for querying and authoring, and why multimodal observation becomes important when BIM tasks are framed as closed-loop embodied agents.

2.1. Image-based generation of space layouts

Plan layout generation refers to arranging rooms and spaces within a floorplan, and has been one of the most extensively explored objectives in architectural design tasks using generative AI (Jang, Roh and Lee, 2025). Early approaches mainly relied on generative adversarial network (GAN)-based pipelines to synthesize plausible layouts within given footprints, often incorporating relational constraints and iterative refinement, as in House-GAN and House-GAN++ (Nauata *et al.*, 2020, 2021). Recent work has shifted toward diffusion-based generation of vector/coordinate layouts, improving geometric consistency (e.g., HouseDiffusion). Conditioning has also expanded beyond fixed footprints to intermediate representations such as bubble-diagram graphs and multimodal inputs that combine graphs with footprint images. For example, Shabani, Hosseini, and Furukawa (2023) demonstrate diffusion-based vector floorplan generation conditioned on graph-formatted bubble diagrams. This line of work has evolved from graph-to-image generation toward richer multimodal inputs that combine graphs with footprint images, aiming to better capture both relational structure and geometric context. In parallel, prompt-guided generation and editing of floor plans has emerged, indicating increasing interest in natural-language conditioning and multi-constraint specification for layout modification tasks. Despite this progress, many image-based floorplan generation pipelines remain difficult to operationalize in BIM environments. Pixel-level outputs require conversion into BIM elements and often lose geometric and semantic fidelity under iterative BIM-to-image round trips, motivating structured representations coupled with executable edits in BIM environment.

2.2. Data Representations and Context for LLM-BIM Workflows

Recent studies have integrated large language models (LLMs) into BIM workflows to enable natural-language access to BIM information and to support BIM tasks such as information retrieval and design authoring operations. A common focus of those studies is regarding how to represent BIM information into LLM-interpretable input and maintaining outputs controllable and executable. Most proposed systems externalize BIM context into structured text-based representations with metadata, such as java script object notation (JSON) or comma separated values (CSV), ontology-aligned triples (e.g., RDF/Turtle), or descriptive text derived from semantic labels.

For authoring-oriented tasks, these representations serve as an intermediate layer that bridges BIM model states and LLM reasoning reflection. For example, Jang et al. (2024) proposed a framework to extract object-oriented programming (OOP)-based BIM instances into a JSON schema and use LLMs to decide detailing actions over the schema, and map outputs back to BIM models through Revit application programming interface (API) operations. Du et al. (2026) represented BIM model generation as LLM-generated imperative code that invokes high-level modeling APIs within the authoring tool. Forth and Borrmann (2024) used BIM-provided semantic labels as textual cues and apply semantic textual similarity to enrich and classify model semantics for downstream simulation preparation. Collectively, these approaches show that text-represented BIM context can support both controllability and executability, especially when the target task can be expressed as discrete attribute retrieval or structured action parameters.

Table 1. Previous studies integrating LLM into BIM systems.

References	BIM format	Representation format	Description
(Zheng and Fischer, 2023)	.rvt	Structured text (.json, .bson)	Converts Revit models via APS/SVF2 into model-derivative JSON (e.g., tree.json, properties.json) and stores them as BSON in MongoDB for retrieval-augmented prompting.
(Jang et al., 2024)	.rvt	Structured text (.json)	Extracts Revit instances into a JSON schema, lets an LLM generate detailing decisions over the schema, and maps outputs back to Revit API authoring actions.
(Chen et al., 2024)	.ifc	Structured text (.ttl)	Transforms IFC into ontology-aligned RDF triples (Turtle) to support LLM-assisted compliance checking over a graph-structured representation.
(Forth and Borrmann, 2024)	.ifc	Descriptive text	Uses semantic textual similarity over IFC entity “Name” attributes (e.g., IfcSpace/IfcMaterialLayer/IfcMaterial names) to support semantic enrichment for downstream simulation preparation.
(Fernandes et al., 2024)	.rvt	Structured (.json, .csv)	Builds a LLM-based assistant by exporting Revit/APS-derived model metadata to JSON and flattening element/property data into CSV for querying and analysis.
(Guo et al., 2025)	.rvt	Structured (.json)	Refines Revit API documentation into a hierarchical JSON retriever and translates user queries into a query-domain-specific language / logical chain to align with domain code functions for retrieval code generation.
(Du et al., 2026)	.vwx	Design parameters for predefined toolset	Represents BIM model generation as LLM-generated imperative code that invokes high-level modeling APIs in the authoring tool.
(Dong et al., 2025)	.rvt	Structured text and view snapshot image	Packages view-related logs (e.g., camera pose, zoom level, view type, selected element ID) together with corresponding Revit view snapshots to provide procedural observation context for multi-agent coordination.
(Deng et al., 2025)	.vwx	Graphical user interface (.jpg, .png)	Provides visual context to LLM agents to conduct BIM authoring along based on given hand-drawn sketch.

For BIM querying and information retrieval, systems similarly transform BIM knowledge into searchable textual indices. Zheng and Fischer (2023) converted Revit models through an SVF2-based pipeline to produce model-derivative JSON (e.g., hierarchy and properties) and store them in a database (e.g., BSON in MongoDB) for retrieval-augmented prompting. Similarly, Fernandes et al. (2024) exported Revit-derived metadata into structured artifacts such as JSON and CSV to enable LLM-based querying over element/property data. Guo et al. (2025) addressed retrieval from another angle by aligning natural-language queries to domain code functions. They restructure Revit API documentation

into a hierarchical JSON format for retrieval and express the intended retrieval process as a DSL/logical chain that supports subsequent code generation.

However, a limitation of purely text-represented BIM context is that it can under-specify the context that BIM users rely on when formulating requests and validating results. In practical authoring, user intent is often grounded in what is currently inspected in the interface (e.g., a specific view, zoom scale, or local geometric congestion), which may not be recoverable from textualized element and property representation alone. Dong et al. (2025) make this gap explicit by capturing an observation package that couples Revit view snapshots with view-metadata logs (e.g., view orientation vectors, eye position, zoom scale, and an image path) to document what users were looking at during coordination tasks. Their workflow highlights that, beyond structured BIM data, incorporating interface-level visual and procedural context can improve an agent’s ability to infer user focus and support capturing procedural knowledge.

These observations connect naturally to agentic and embodied AI, where an agent iteratively alternates between perceiving the environment and acting through an interface, using feedback to refine its next decisions. ReAct is a representative pattern in this direction, where it explicitly prompts an LLM to generate interleaved reasoning traces and environment actions, so the model can update high-level plans based on newly observed outcomes rather than relying on a single-shot decision (Yao *et al.*, 2022). Recent work on agent integration into BIM system also demonstrates the potential of this direction (Deng *et al.*, 2025). In parallel, scalable instructable multiworld agent (SIMA) 2 demonstrates that “embodied agent” can be realized in virtual environments. The embodied agent perceives the world through visual frames and acts through a human-like keyboard–mouse action space, without privileged access to underlying state, while still supporting goal-directed reasoning and dialogue (SIMA Team and Google Deepmind, 2025). BIM authoring environments are a particularly good substrate for bringing these ideas into design workflows because the action space is well-defined and executable (e.g., element relocation, parameter edits through APIs), and outcomes can be verified against explicit constraints. In this setting, aligned multimodal observation becomes central where structured BIM-derived representations enable precise constraint-aware execution, while interface-consistent visual evidence (e.g., the current view context that a user relies on) reduces ambiguity in intent interpretation and strengthens verification and recovery across iterative authoring loops. Which can collaboratively support the closed-loop behavior emphasized in ReAct interaction and embodied interfaces similar to SIMA.

2.3. Research gaps

From the above literature, two research gaps are identified. First, there remains a grounding gap between image-centric layout generation and executable BIM authoring. Image-based generative methods offer rich visual context for layout reasoning, yet their outputs do not translate directly into actionable BIM modifications. Conversely, existing LLM–BIM integrations enable execution through structured textual representations, but they are weakly grounded in the visual and topological context that users typically reference. This points to the lack of a unified mechanism to derive aligned multimodal observations (e.g., view-based visual cues and layout topology) from BIM models for intent understanding. Second, prior BIM-oriented agents seldom realize a fully operational closed-loop editing workflow in which re-perception and constraint checking are integral to robust multi-step modification. Although recent studies suggest that interface-level artifacts such as view snapshots and logs can reveal user focus, they have not been systematically leveraged as aligned observations to support constraint-aware BIM editing under practical scenarios. Motivated by these gaps, this paper investigates the following research questions:

(1) How can aligned topological and visual context be extracted from BIM and represented as observations for an embodied BIM-editing agent?

(2) To what extent does augmenting structured topology with aligned visual context improve interactive space layout modification performance under realistic constraints, compared to topology-only baselines?

3. Proposed Method

This study proposes a multimodal context–augmented embodied agent method for interactive BIM space layout modification (Figure 2). The method is composed of three modules: (1) a rule-based translation module, (2) a relocation agent module, and (3) an evaluation agent module. These modules jointly realize an embodied-agent loop. In this loop, the agent must be grounded in a task environment that exposes the current layout state, while its decisions must remain executable on the underlying BIM representation. The rule-based translation module provides this grounding by converting BIM into a task-relevant structured topology representation and an aligned visual context. The relocation agent module then uses these contexts to produce an explicit relocation plan that targets existing BIM element IDs. Finally, the evaluation agent module judges whether the resulting candidate satisfies the given requirements and constraints, and returns structured feedback that can drive iterative refinement.

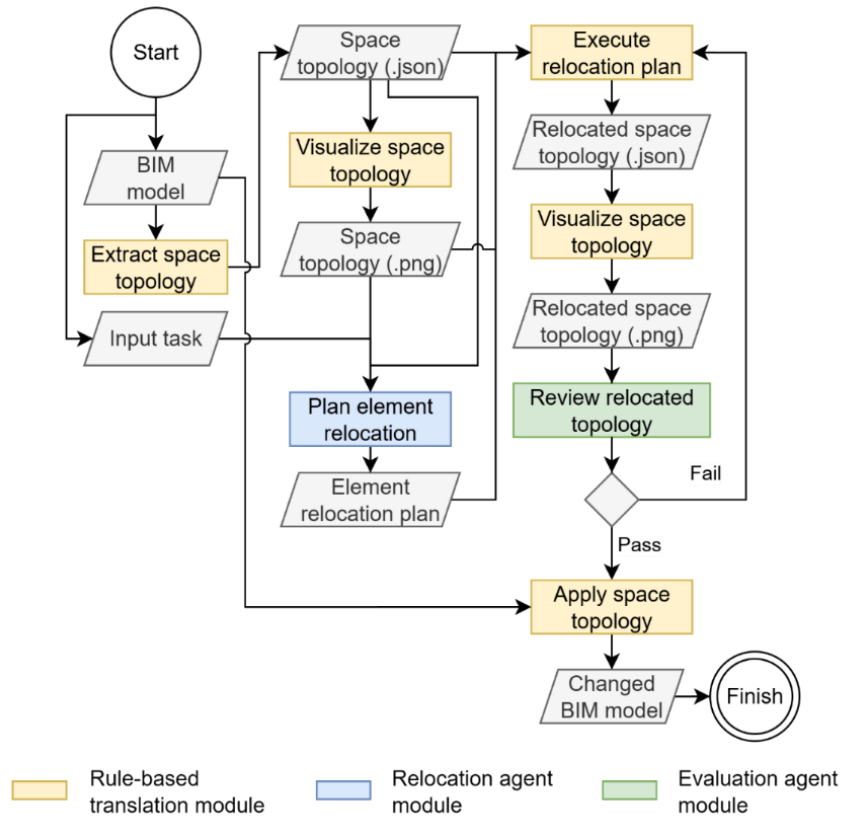


Figure 2. Proposed multimodal context–augmented embodied agent method for interactive BIM space layout modification.

3.1. Rule-based translation module

The rule-based translation module bridges the BIM environment and the embodied agents by producing agent consumable visual context and by ensuring that agent decisions can be applied back to the layout state. Figure 3 summarizes the two aligned contexts produced from the same BIM derived layout state. First, the module extracts a spatial topology with geometric placeholders in JSON that includes rooms and their boundary defining wall segments. Each wall segment is tracked with stable element IDs and

a compact geometric proxy such as the start point and end point of its location curve, and each room stores the list of bounding element IDs so the agent can directly reference manipulable boundaries. Second, the module renders an element annotated floorplan from the same structured state using rule-based algorithm. The rendering depicts only the boundary segments relevant to relocation and place the element IDs next to the corresponding segments, so the agent can ground target selection in the same IDs used for execution.

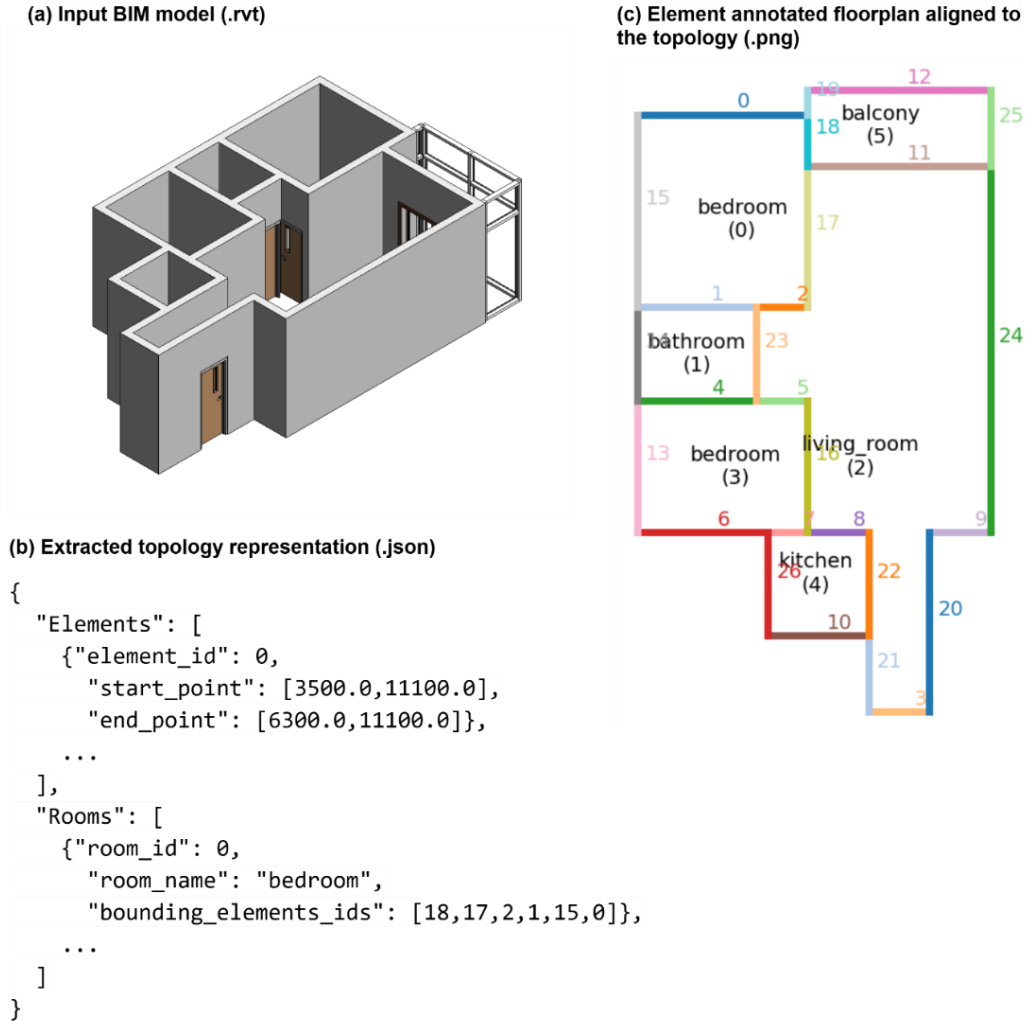


Figure 3. Rule-based translation from BIM to aligned topology and visual context.

After the relocation agent proposes a plan, the translation module applies the plan deterministically to the structured layout state. The plan references only existing element IDs and specifies geometric changes using either updated endpoints or a displacement vector. To preserve boundary consistency, the module enforces rule-based connectivity maintenance by propagating updated endpoint coordinates to any other boundary segments that share the same junction. This deterministic execution closes the loop and isolates the agent’s contribution to planning and constraint handling under a fixed visual context.

3.2. Relocation agent module

The relocation agent module is responsible for planning executable layout modifications under the aligned contexts produced by the rule-based translation module. Here, aligned context means (1) an extracted space topology representation in JSON and (2) an element-annotated floorplan image

rendered from that same topology. Given a user request and associated constraints, the agent consumes these two aligned inputs to ground target selection and to avoid ambiguity between what is seen in the view and what is executed in the structured state.

Using the aligned topology and annotated floorplan, the agent produces an explicit relocation plan that targets only existing boundary element IDs and can be deterministically applied by the translation module. Each plan step specifies which element to move and how, either by proposing updated endpoints or by providing a displacement vector. When needed, it also specifies connected adjustments so that junctions shared by multiple boundary segments remain coherent after execution. The agent does not modify the environment directly. Instead, it proposes a plan, receives structured feedback from the evaluation agent after deterministic execution, and revises the plan in subsequent iterations.

The relocation agent is instructed to operate only on the provided aligned context and to output schema-conforming plans. The instruction emphasizes using only existing element IDs from the topology, limiting actions to boundary relocation and endpoint edits, and expressing geometric changes in an executable form that the rule-based translation module can apply without additional interpretation.

3.3. Evaluation agent module

The evaluation agent module functions as a strict checker within the proposed method. It assesses whether the candidate layout state produced by relocation agent module satisfies the task intent and remains compliant with constraints. The evaluation agent receives the task prompt and constraints together with the original structured topology state, the updated structured topology state, and the relocation plan that generated the update. It returns a structured pass or fails decision with issue descriptions and corrective suggestions that can directly guide the next planning iteration.

Requirement satisfaction is assessed by verifying that the target room is modified in the intended direction, such as expansion of the living room under the allowed operations (Figure 1). Constraint satisfaction is assessed by checking for prohibited side effects, such as reductions in other rooms or violations of the building boundary, as specified in the task prompt. The checker also assesses common robustness of output such as whether the plan referenced only existing IDs and whether boundary connectivity remains coherent after the applied updates.

The evaluation agent is instructed to behave as a deterministic supervisor and to output only schema-constrained structured results. The instruction requires explicit comparison between original and updated topology states under the stated constraints, and it requires concrete failure reasons and actionable suggestions rather than proposing new edits.

4. Validation

The proposed framework is validated on 15 single-floor residential BIM models reconstructed from the RPLAN dataset (Wu *et al.*, 2019). Each model is converted into a single-floor BIM model composed of rooms and boundary walls with doors. Boundary walls are tracked with stable element IDs and geometric proxies parsed from their location curves (Figure 3b). When multimodal context is enabled, the same structured layout state is paired with an element-annotated floorplan rendering in which displayed wall instances are aligned to the same element IDs (Figure 3b and Figure 3c).

Validation evaluates the three steps of the generalized generative-AI-augmented BIM framework (i.e., structure, execute, and check) under an iterative refinement loop. In each iteration, the relocation agent structures an explicit relocation plan from the current observation. The plan is executed through deterministic rule-based updates to the layout state, and the resulting candidate is checked against task

requirements and constraints by the evaluation agent. The loop terminates when the checker passes the candidate or when the iteration reaches maximum threshold (considered as practical failure).

Two task prompts are used to test constraint understanding under the same high-level objective of expanding the living room (as illustrated in Figure 1). The first task asks for expansion without affecting other spaces. The second asks for expansion while maintaining the building boundary. These prompts evaluate whether the agent selects appropriate boundaries and relocation targets under different constraint settings. Observation modality is also ablated, where the baseline provides only structured layout representation while the proposed condition adds the aligned annotated floorplan image.

All agent-driven modules utilized in the validation run with a fixed pinned model from OpenAI ‘gpt-5.1-2025-11-13’ without fine-tuning. The relocation agent produces relocation plans under a JSON schema constraint, and the evaluation agent outputs structured pass/fail decisions with issues and suggestions under a separate schema. The relocation agent runs with moderate exploration (temperature 0.4), while the evaluation agent is fixed for stable compliance decisions (temperature 0.0). Each run uses a maximum budget of 10 iterations and terminates immediately upon success. At every iteration, intermediate artifacts are recorded for interpretation of the results.

Final task performance is assessed by manual review of the last candidate produced by the relocation agent. For each case, the authors label the result as pass or fail based on whether the task prompt is satisfied. For the task without affecting other spaces, a pass requires that the living room expands while surrounding spaces remain unaffected. For the task of maintaining the building boundary, a pass requires that the boundary is preserved while the living room expands by adjusting interior boundaries. Based on the authors’ judgment, the relocation agent task success rate is computed across all cases and ablation settings. In addition, discrepancies between the evaluation agent decisions and the author labels are recorded to estimate the reliability of the checker. Instruction prompts, test dataset, and JSON schemas are available at: <https://github.com/suhyung-research/Multi-Modal-Context-Augmented-Embodied-Agent-for-Interactive-BIM-Space-Layout-Modification>.

5. Results and Discussions

Figure 4 compares two observation settings, structured topology only and structured topology with aligned visual context as multimodal input, evaluated across 15 space layouts. Overall success rates remain moderate in all settings, but adding aligned visual context yields consistent, incremental gains across both tasks.

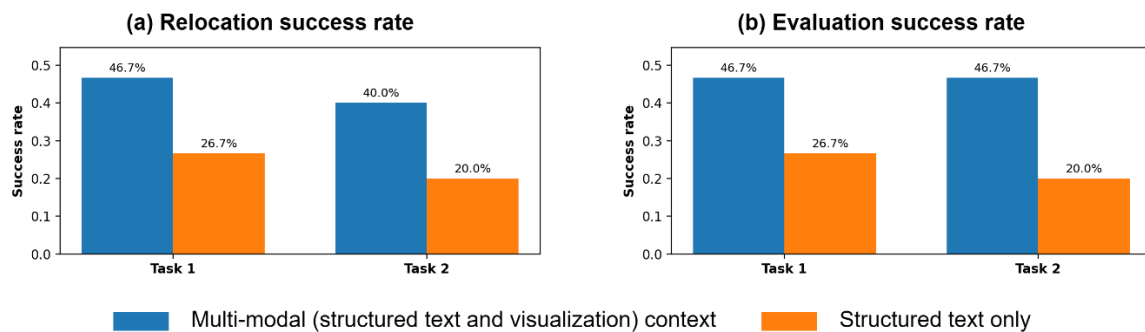


Figure 4. Effect of context modality on success rates in the embodied relocation loop.

In Figure 4a, multimodal context improves relocation success in both tasks. For Task 1, relocation success increases from 26.7% (4 out of 15) under structured-only context to 46.7% (7 out of 15) under multimodal context. For Task 2, success increases from 20.0% (3 out of 15) to 46.7% (7 out of 15). These results suggest that, even when the overall task remains challenging, providing an element-aligned layout image alongside the structured representation can help the relocation agent make slightly more reliable boundary-relocation decisions.

Figure 4b reports evaluation success as the agreement between the evaluation agent’s pass decision and the human judgement. A similar pattern is observed. For Task 1, agreement rises from 26.7% (4 out of 15) to 46.7% (7 out of 15) when multimodal context is used. For Task 2, agreement increases from 20.0% (3 out of 15) to 40.0% (6 out of 15). Evaluation results remain far from perfect, indicating that verification is still a non-trivial component of the loop, but multimodal context again provides a small benefit. The only noticeable divergence between Figure 4a and Figure 4b occurs for Task 2 under multimodal context, where relocation success is 46.7% (7 out of 15) while evaluation agreement is 40.0% (6 out of 15). This indicates that at least one case was judged successful by the human evaluator while the evaluation agent did not produce a matching pass outcome, reflecting sensitivity to termination and verification criteria such as iteration budget or constraint checking. This reinforces that, in an embodied-agent loop, the reported task outcome depends not only on whether a plausible relocation is produced, but also on how the final verification step operationalizes success.

In summary, while the absolute success rates are not high, the results show that multimodal context provides a modest, context-dependent improvement in providing task context required for evaluation, supporting the value of combining structured BIM-derived textual context with aligned task-relevant visual context for interactive space layout modification.

6. Conclusions

This paper presented a multi-modal context-augmented embodied agent method for interactive BIM space layout modification. The method addresses the integration gap between image-centric layout reasoning and executable BIM editing by grounding the agent using two aligned observations derived from the same BIM-based layout state. It provides a JSON topology state with stable element IDs and compact geometric proxies, and a floorplan view annotated with the same IDs. This alignment supports the agent connects what it sees with what it can directly edit. Within an iterative loop, a relocation agent produces schema-constrained boundary relocation plans, a rule-based execution step applies the plans deterministically while preserving junction consistency, and an evaluation agent checks requirements and constraints to guide refinement.

Validation on 15 residential layouts with two living-room expansion tasks shows that adding aligned visual context yields consistent but modest improvements over a structured-only baseline, while absolute success rates remain moderate. Relocation success increases from 26.7% to 46.7% in Task 1 and from 20.0% to 46.7% in Task 2 when multimodal context is provided. Evaluation success shows the same trend, with agreement increasing from 26.7% to 46.7% in Task 1 and from 20.0% to 40.0% in Task 2. These results suggest two findings: i) multimodal grounding reduces ambiguity in target selection and constraint interpretation, thereby providing a more viable pathway for AI agents to perform direct, vision-aware BIM authoring; but at the same time ii) the tasks remain challenging due to compounded failure sources across planning and verification. We also observed that the reported task outcome is sensitive to the verification and termination behavior of the evaluation module, indicating that final success depends on both edit generation and how verification operationalizes success.

The study has several limitations that motivate future work. First, the action space is restricted to boundary relocation and endpoint edits, which limits the system’s ability to resolve cases that require topology-changing operations such as splitting, merging, or reassigning spaces. Second, the current

topology representation and proxy geometry are intentionally compact, which improves executability but may omit contextual cues needed to satisfy nuanced constraints, such as subtle impacts on adjacent rooms or circulation. Third, verification is performed by an LLM-based evaluation agent rather than a fully rule-based geometric and constraint solver, which introduces uncertainty in reliability and can lead to mismatches with manual assessment. Fourth, the validation scope is limited to 15 residential layouts and two task prompts, and broader generalization across building types, multi-floor models, and richer constraint sets remain to be tested.

Future work will extend the framework in three directions. The first direction is expanding the action primitives to support topology-changing edits and higher-level operators, enabling the agent to resolve a wider set of realistic modification requests. The second direction is strengthening verification by combining an evaluation agent with deterministic rule-based checks, including explicit overlap detection, boundary preservation tests, and room-area and adjacency constraints, and then systematically analyzing which constraint types are handled well and which systematically fail. The third direction is improving multimodal observation and alignment, including richer view context, better abstraction of local geometric congestion, and tighter coupling between visual evidence and structured state to reduce ambiguity across iterative steps. Together, these directions aim to increase robustness, explainability of failures, and scalability of embodied agent strategies for BIM model editing under realistic design constraints.

7. Acknowledgements

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Ministry of Science and ICT (Grant RS-2025-00522484).

8. References

- Cao, Y., Abdul Aziz, A. and Mohd Arshard, W.N.R. (2025) “Stable diffusion in architectural design: Closing doors or opening new horizons?,” *International Journal of Architectural Computing*, 23(2), pp. 339–357. Available at: <https://doi.org/10.1177/14780771241270257>.
- Chen, N. *et al.* (2024) “Automated building information modeling compliance check through a large language model combined with deep learning and ontology,” *Buildings*, 14(7), p. 1983.
- Choi, S. *et al.* (2024) “Generative architectural plan drawings for early design decisions: data grounding and additional training for specific use cases,” *Architectural Engineering and Design Management*, 0(0), pp. 1–21. Available at: <https://doi.org/10.1080/17452007.2024.2445033>.
- Deng, Z. *et al.* (2025) “BIMgent: Towards Autonomous Building Modeling via Computer-use Agents,” in. *ICML 2025 Workshop on Computer Use Agents*. Available at: <https://openreview.net/forum?id=vQCFwXAl2A> (Accessed: February 6, 2026).
- Dong, Y. *et al.* (2025) “AI BIM coordinator for non-expert interaction in building design using LLM-driven multi-agent systems,” *Automation in Construction*, 180, p. 106563.
- Du, C. *et al.* (2026) “Text2BIM: Generating Building Models Using a Large Language Model-Based Multiagent Framework,” *Journal of Computing in Civil Engineering*, 40(2), p. 04025142. Available at: <https://doi.org/10.1061/JCCEE5.CPENG-6386>.

Fernandes, D. *et al.* (2024) “A gpt-powered assistant for real-time interaction with building information models,” *Buildings*, 14(8), p. 2499.

Forth, K. and Borrmann, A. (2024) “Semantic enrichment for BIM-based building energy performance simulations using semantic textual similarity and fine-tuning multilingual LLM,” *Journal of Building Engineering*, 95, p. 110312.

Guo, P. *et al.* (2025) “Advancing BIM information retrieval with an LLM-based query-domain-specific language and library code function alignment system,” *Automation in Construction*, 178, p. 106374.

Jang, S. *et al.* (2024) “Automated detailing of exterior walls using NADIA: Natural-language-based architectural detailing through interaction with AI,” *Advanced Engineering Informatics*, 61, p. 102532. Available at: <https://doi.org/10.1016/j.aei.2024.102532>.

Jang, S., Roh, H. and Lee, G. (2025) “Generative AI in architectural design: Application, data, and evaluation methods,” *Automation in Construction*, 174, p. 106174. Available at: <https://doi.org/10.1016/j.autcon.2025.106174>.

Lee, G., Jang, S. and Hyun, S. (2024) “A Generalized LLM-Augmented BIM Framework: Application to a Speech-to-BIM system,” in *Proceedings of the 41st International Conference of CIB W78*. Marrakech, Morocco.

Nauata, N. *et al.* (2020) “House-GAN: Relational Generative Adversarial Networks for Graph-Constrained House Layout Generation,” in A. Vedaldi *et al.* (eds.) *Computer Vision – ECCV 2020*. Cham: Springer International Publishing (Lecture Notes in Computer Science), pp. 162–177. Available at: https://doi.org/10.1007/978-3-030-58452-8_10.

Nauata, N. *et al.* (2021) “House-gan++: Generative adversarial layout refinement network towards intelligent computational agent for professional architects,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13632–13641. Available at: http://openaccess.thecvf.com/content/CVPR2021/html/Nauata_House-GAN_Generative_Adversarial_Layout_Refinement_Network_towards_Intelligent_Computational_Agent_CVPR_2021_paper.html (Accessed: December 14, 2025).

Shabani, M.A., Hosseini, S. and Furukawa, Y. (2023) “Housediffusion: Vector floorplan generation via a diffusion model with discrete and continuous denoising,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 5466–5475. Available at: http://openaccess.thecvf.com/content/CVPR2023/html/Shabani_HouseDiffusion_Vector_Floorplan_Generation_via_a_Diffusion_Model_With_Discrete_CVPR_2023_paper.html (Accessed: December 14, 2025).

SIMA Team, G. and Google Deepmind (2025) *SIMA 2: A Generalist Embodied Agent for Virtual Worlds*. Google DeepMind.

Wu, W. *et al.* (2019) “Data-driven interior plan generation for residential buildings,” *ACM Transactions on Graphics*, 38(6), pp. 1–12. Available at: <https://doi.org/10.1145/3355089.3356556>.

Yao, S. *et al.* (2022) “React: Synergizing reasoning and acting in language models,” in *The eleventh international conference on learning representations*. Available at: https://openreview.net/forum?id=WE_vluYUL-X (Accessed: December 15, 2025).

Zheng, J. and Fischer, M. (2023) “Dynamic prompt-based virtual assistant framework for BIM information search,” *Automation in Construction*, 155, p. 105067.